7CS039 Statistics for AI & Data Science

Week 6 Introduction to Statistical Classification

Dr. Liam Naughton

School of Mathematics & Computer Science
UoW

Introduction

- Classification is a common problem encountered as a data scientists.
- The central aim of a classifier is to predict the category an object belongs to based on the features or properties that object has.
- This can be as simple as classifying an email as spam or not or as classifying mushroom as poisonous or not (we'll study this example later).

Classification

Classification is concerned with predicting the target class for an object based on the features of that object. The possible classes are already known and the algorithm needs to identify which class the object belongs to.

Terminology

- **Classifier:** A classifier is an algorithm that classifies the input data into output categories.
- **Classification model:** A classification model is a model that uses a classifier to classify data objects into various categories.
- **Feature:** A feature is a measurable property of a data object.

- Classification is a form of supervised learning where the response variable is categorical, as opposed to numeric.
- Our goal is to find a rule, algorithm, or function which takes as input a *feature vector*, and outputs a category which is the true category **as often as possible**.
- Our first classifier will use the ISLR data set.

```
> install.packages("ISLR")
> library(ISLR)
> library(tibble)
```

```
>
> library(ISLR)
>
> library(tibble)
> as_tibble(Default)
# A tibble: 10,000 × 4
   default student balance income
   <fct>   <fct>     <dbl>  <dbl>
 1 No      No         730. 44362.
 2 No      Yes        817. 12106.
 3 No      No        1074. 31767.
 4 No      No         529. 35704.
 5 No      No         786. 38463.
 6 No      Yes        920.  7492.
 7 No      No         826. 24905.
 8 No      Yes        809. 17600.
 9 No      No        1161. 37469.
10 No      No           0  29275.
# … with 9,990 more rows
>
```

The `Default` dataset is a simulated data set containing information on ten thousand customers. The aim here is to predict which customers will default on their credit card debt.

The dataset has the following format

- `default` A factor with levels No and Yes indicating whether the customer defaulted on their debt
- `student` A factor with levels No and Yes indicating whether the customer is a student
- `balance` The average balance that the customer has remaining on their credit card after making their monthly payment
- `income` Income of customer

Note that the variables `default` and `student` are factor variables.

The first thing we do is split our data into a training set and a test set. We'll use 50% for each.

```
> set.seed(20)
> train_index = sample(nrow(Default), 5000)
> train_default = Default[train_index, ]
> test_default = Default[-train_index, ]
```

Now will do some quick visualizations to help us suggest some classification rules. To do this we will use the `caret` package.

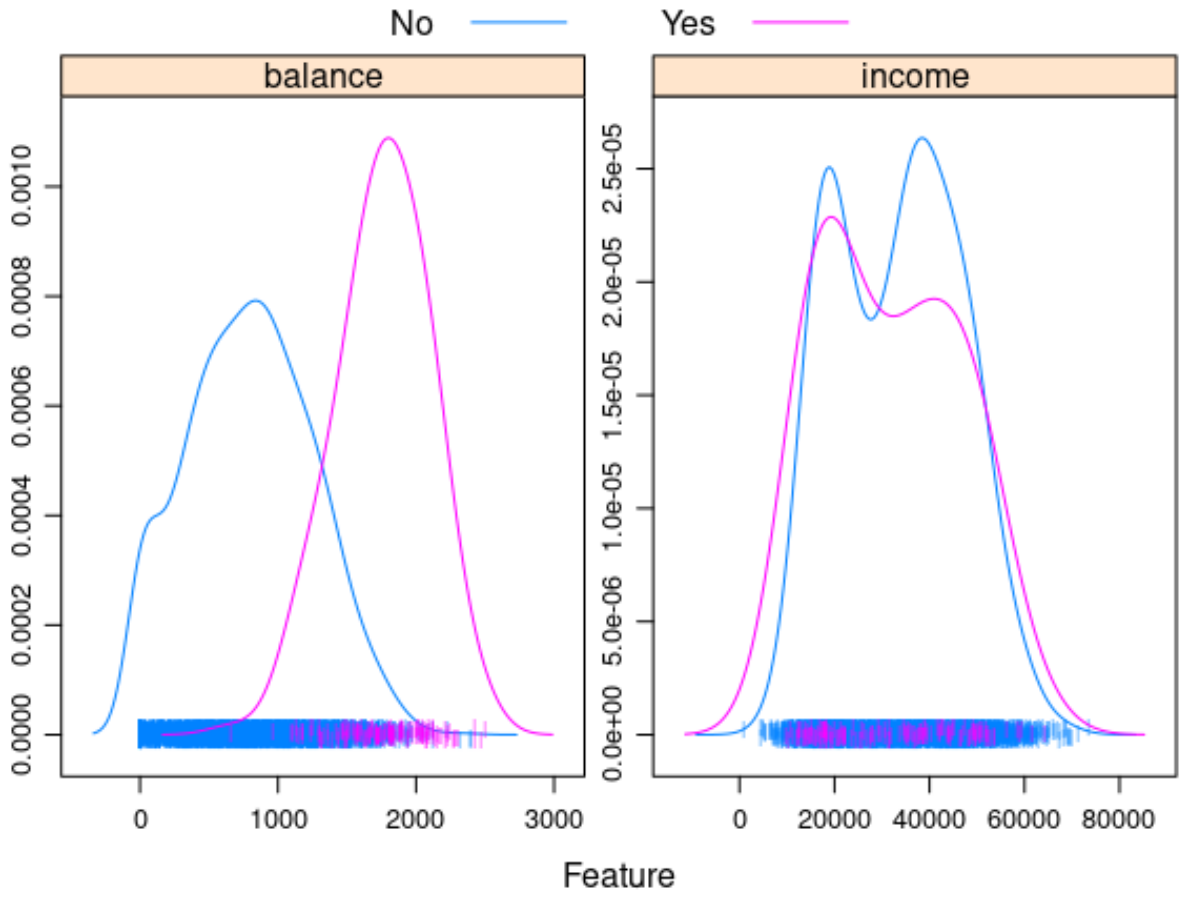
```
> library(caret)
```

Now we'll use a density plot to explore properties of the numeric predictors.

```
featurePlot(x=train_default[,c("balance", "income")],
        y = train_default$default,
        plot = "density",
        scales = list(x = list(relation = "free"),
                y = list(relation = "free")),
        adjust = 1.5,
        pch = "|",
        layout = c(2, 1),
        auto.key = list(columns = 2))
```

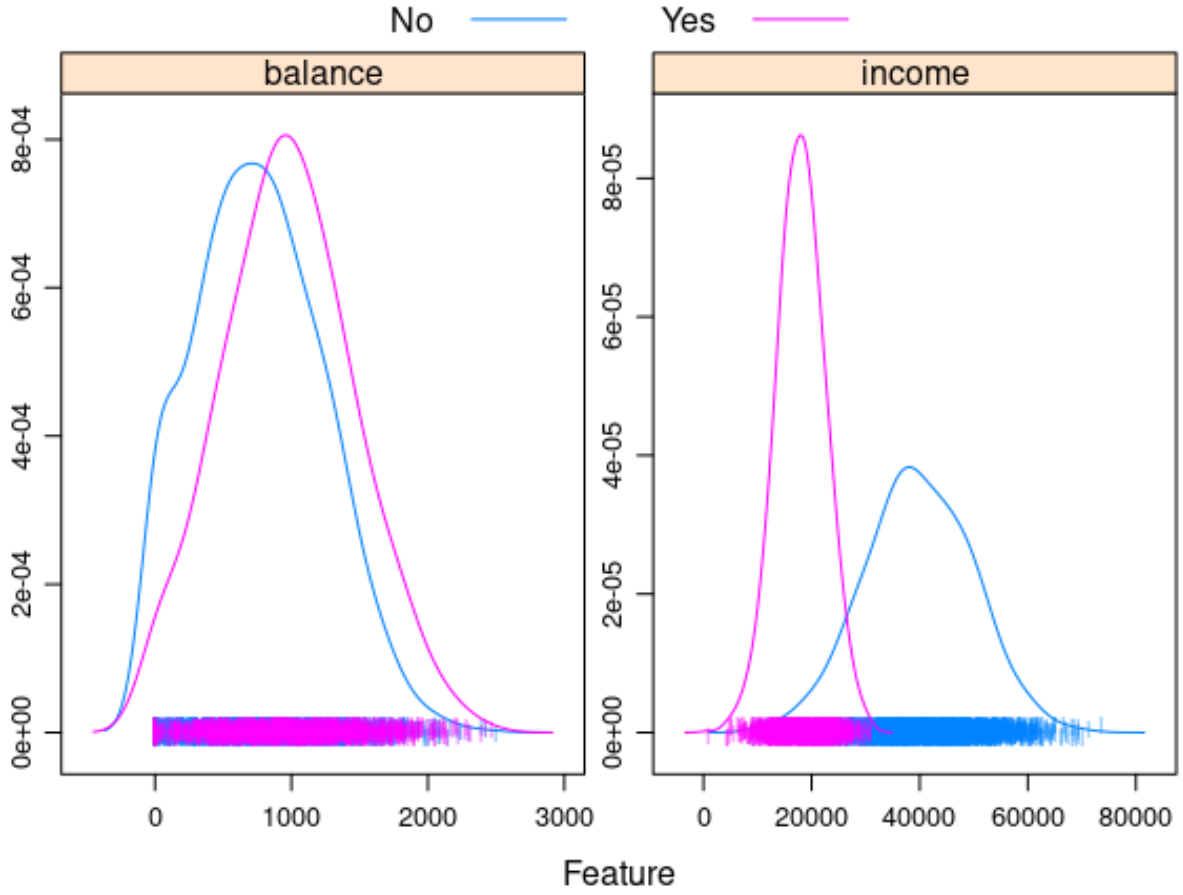
In this function we have

- x is a data frame containing only numeric predictors.
- y is the response variable. It needs to be a factor variable.
- `plot` specifies the type of plot, here it's `density`.
- `scales` defines the scale of the axes for each plot. By default, the axis of each plot would be the same, which often is not useful, so the arguments here, a different axis for each plot, will almost always be used.
- `adjust` specifies the amount of smoothing used for the density estimate. (you can play around with this)
- `pch` specifies the plot character used for the bottom of the plot.
- `layout` places the individual plots into rows and columns. It is given as column, row.
- `auto.key` defines the key at the top of the plot. The number of columns should be the number of categories.


No
Yes
balance
income
0.0000
0.0002
0.0004
0.0006
0.0008
0.0010
0.0e+00
5.0e-06
1.0e-05
1.5e-05
2.0e-05
2.5e-05
0
1000
2000
3000
0
20000
40000
60000
80000
Feature

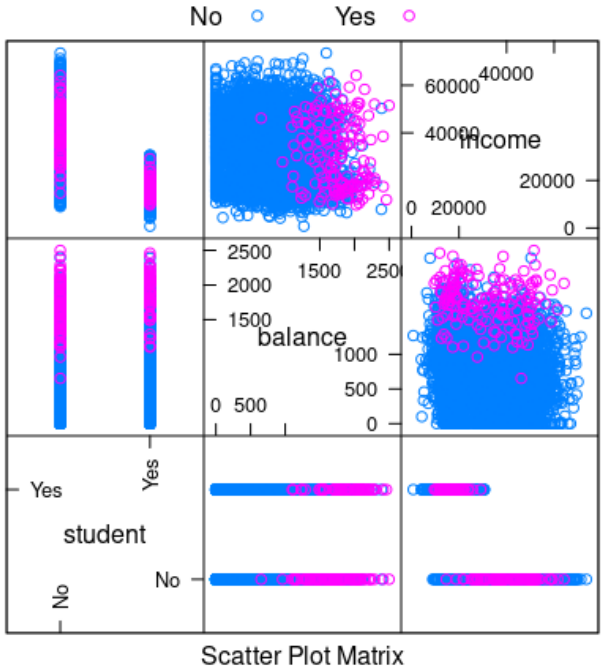
The income variable by itself is not particularly useful predictor of whether or not someone will default. However, there seems to be a big difference in default status at a balance of about 1400. We will use this information shortly. Now let's explore the relationship with the variable `student` as the response.

```
featurePlot(x =train_default[,c("balance", "income")],
            y = train_default$student,
            plot = "density",
            scales =list(x = list(relation = "free"),
                         y = list(relation = "free")),
            adjust = 1.5,
            pch = "|",
            layout = c(2, 1),
            auto.key = list(columns = 2))
```



We can see that students have a much lower income (perhaps not surprising). Now let's plot pairs of variables.

```
featurePlot(x = train_default[, c("student",
            "balance", "income")],
            y = train_default$default,
            plot = "pairs",
            auto.key = list(columns = 2))
```



This plot also suggests that using `balance` to create a classifier will be more useful than `income`.

Our First Classifier

We already noted that whether or not someone defaults seems to change when the balance variable is around 1400. Let's (naively) design our classifier as follows

$$\text{Classify(balance)} = \begin{cases} \text{Yes} & \text{if balance} > 1400 \\ \text{No} & \text{if balance} \leq 1400 \end{cases}$$

This means that we will predict an individual is a defaulter if their `balance` is above 1400 and predict that they are not otherwise. Let's see how our classifier performs. First we write a simple function to do the classification for us according to the rule above.

```
> MyFirstClassifier=function(x,boundary,above = 1,
    below = 0) {
    ifelse(x > boundary, above, below)
}
```

Then we put this to use on the training and test sets.

```
> training_predictor = MyFirstClassifier(x =
      train_default$balance, boundary = 1400,
      above = "Yes", below = "No")
```

```
>  MyFirstClassifier = function(x, boundary, above = 1, below = 0) {
+     ifelse(x > boundary, above, below)
+ }
> training_predictor = MyFirstClassifier(x = train_default$balance,
+                                        boundary = 1400, above = "Yes", below = "No")
> head(training_predictor)
[1] "No"  "No"  "No"  "No"  "Yes" "No"
>
```

```
> test_predictor=MyFirstClassifier(x =
        train_default$balance,
        boundary = 1400, above = "Yes",
        below = "No")
```

```
> test_predictor = MyFirstClassifier(x = train_default$balance,
+                                    boundary = 1400, above = "Yes", below = "No")
> head(test_predictor, n=10)
 [1] "No"  "No"  "No"  "No"  "Yes" "No"  "No"  "Yes" "No"  "Yes"
>
```

So there we have it. A simple crude classifier which will take an input vector and predict whether or not the person will default.

Is it any good though?

One way to measure how good our classifier is, is to make a table of the predictions and the true default numbers.

```
(trainingset_table = table(predicted
    = training_predictor,
    actual = train_default$default))
```

```
> (trainingset_table = table(predicted = training_predictor, actual = train_default$defau
lt))
         actual
predicted   No  Yes
      No  4378   22
      Yes  467  133
>
```

```
> (testset_table = table(predicted = test_predictor,
    actual = test_default$default))
```

```
> (testset_table = table(predicted = test_predictor, actual = test_default$default))
         actual
predicted   No  Yes
      No  4242  158
      Yes  580   20
>
```

These tables are usually referred to as a **confusion matrix**. The entries in the table represent (and be careful about this)

True no	False no
False yes	True yes

Meaning

- that the number in the top left is the number or cases correctly identified as no,
- the number in the bottom right is the number of classes correctly classified as yes.
- The number in the bottom left is the number of cases classified incorrectly as defaulters and the number in the top right is the number of cases classified incorrectly as not having defaulted.
- In statistics we think if these as type I and type II errors.

- The `caret` package includes a handy function for confusion matrices.

```
> training_con_mat =
    confusionMatrix(trainingset_table,
    positive = "Yes")
> training_con_mat
```

```
> training_con_mat = confusionMatrix(trainingset_table, positive = "Yes")
> training_con_mat
Confusion Matrix and Statistics

         actual
predicted   No  Yes
      No  4378   22
      Yes  467  133

               Accuracy : 0.9022
                 95% CI : (0.8936, 0.9103)
    No Information Rate : 0.969
    P-Value [Acc > NIR] : 1

                  Kappa : 0.3188

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.8581
            Specificity : 0.9036
         Pos Pred Value : 0.2217
         Neg Pred Value : 0.9950
             Prevalence : 0.0310
         Detection Rate : 0.0266
   Detection Prevalence : 0.1200
      Balanced Accuracy : 0.8808

       'Positive' Class : Yes
```

```
> testset_con_mat =
    confusionMatrix(testset_table,
    positive = "Yes")
> testset_con_mat
```

```
> testset_con_mat = confusionMatrix(testset_table, positive = "Yes")
> testset_con_mat
Confusion Matrix and Statistics

         actual
predicted   No  Yes
      No  4242  158
      Yes  580   20

               Accuracy : 0.8524
                 95% CI : (0.8423, 0.8621)
    No Information Rate : 0.9644
    P-Value [Acc > NIR] : 1

                  Kappa : -0.0037

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.11236
            Specificity : 0.87972
         Pos Pred Value : 0.03333
         Neg Pred Value : 0.96409
             Prevalence : 0.03560
         Detection Rate : 0.00400
   Detection Prevalence : 0.12000
      Balanced Accuracy : 0.49604

       'Positive' Class : Yes

>
```

The information included in the output tells us how good our classifier is. There is a lot of information here but the most important parts are

- Accuracy - the proportion of predicted classes which are correct.
- Sensitivity - the proportion of true positives classified as positive (the true positive rate if you like)
- Specificity - the proportion of true negatives classified as negative (the true negative rate if you like)

That's enough for this session. We'll explore this further in the workshop by looking at the `OneR` package.